

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 10 – State, Hooks (useState, useEffect), dan Conditional Rendering

Nama Mahasiswa : Wianda Sukandari
NIM : 23051430002
Kelas : A
Tanggal Praktikum : 26 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 10)
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	State Management, Lifecycle Hooks, dan Rendering Bersyarat
Durasi Praktikum	340 Menit
Tanggal Praktikum	26 April 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React.js	React 18+	Library utama untuk membangun

			antarmuka aplikasi web
5	Project React dari pertemuan 9		

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini, konsep utama yang dipelajari adalah penggunaan state dan hooks pada React, khususnya `useState` dan `useEffect`. State merupakan data yang dapat berubah di dalam sebuah komponen dan digunakan untuk mengatur tampilan yang bersifat dinamis. Berbeda dengan props yang hanya menerima data dari luar, state dapat diubah langsung oleh komponen itu sendiri sehingga React akan melakukan re-render secara otomatis saat nilainya berubah.

Hook `useState` digunakan untuk membuat dan mengelola state pada komponen fungsional. Dengan hook ini, programmer dapat menyimpan nilai seperti angka counter, status tombol, atau data input pengguna. Ketika nilai state diperbarui menggunakan fungsi setter, tampilan antarmuka akan ikut berubah tanpa perlu me-refresh halaman secara manual. Hal ini membuat aplikasi menjadi lebih interaktif dan efisien.

Selain itu, `useEffect` digunakan untuk menjalankan proses tertentu saat komponen pertama kali ditampilkan (mount), saat terjadi perubahan data (update), maupun saat komponen dihapus (unmount). Dalam praktikum ini, penerapan state terlihat pada pembuatan komponen counter sederhana yang menampilkan jumlah angka dan dapat bertambah saat tombol diklik. Konsep ini menjadi dasar penting dalam pengembangan aplikasi web modern berbasis React.

D.2 Keterkaitan dengan Teknik Industri

Materi React tentang state dan hooks memiliki keterkaitan dengan Teknik Industri terutama dalam pengembangan sistem informasi yang mendukung proses operasional perusahaan. Teknik Industri tidak hanya berfokus pada proses produksi, tetapi juga pada efisiensi sistem kerja, pengolahan data, dan pengambilan keputusan berbasis teknologi. Penggunaan React dapat membantu dalam membangun dashboard interaktif yang menampilkan data produksi secara real-time.

Contohnya, pada perusahaan manufaktur, sistem monitoring produksi dapat dibuat menggunakan React untuk menampilkan jumlah barang yang telah diproduksi, target produksi harian, serta status mesin secara langsung. Dengan penggunaan state, data produksi dapat terus diperbarui secara otomatis tanpa perlu memuat ulang halaman, sehingga supervisor dapat memantau kondisi lapangan dengan lebih cepat dan akurat.

Selain itu, React juga dapat digunakan dalam sistem manajemen persediaan, penjadwalan kerja, maupun evaluasi performa operator. Dalam konteks Teknik Industri, teknologi ini membantu meningkatkan efisiensi proses bisnis, mengurangi kesalahan manual, serta mendukung pengambilan keputusan yang lebih tepat berdasarkan data yang selalu terbaru.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Counter Produksi

Pada langkah ini saya membuat komponen CounterProduksi.jsx untuk menghitung jumlah produksi secara sederhana menggunakan React useState. Nilai awal produksi dibuat 0 dan target produksi ditentukan 100 unit. Dengan useState, jumlah produksi bisa berubah secara otomatis saat tombol ditekan.

Saya juga menambahkan tombol +1 Unit untuk menambah jumlah produksi dan tombol Reset Shift untuk mengembalikan jumlah produksi ke 0. Selain itu, dibuat juga tampilan status yang akan berubah secara otomatis. Jika jumlah produksi sudah mencapai target, akan muncul pesan "Target Tercapai!", sedangkan jika belum maka akan muncul pesan "Produksi Berjalan...". Jadi, komponen ini membantu memantau proses produksi dengan lebih mudah dan interaktif.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React, { useState } from 'react';
function CounterProduksi() {
  // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah' fungsi untuk
  mengubahnya
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  // Fungsi menambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };
  // Fungsi reset
  const reset = () => {
    setJumlah(0);
  };
  return (
    <div className="text-center p-4 border rounded bg-light">
      <h3>Simulasi Hitung Produk</h3>
      <h1 className="display-4">{jumlah}</h1>
      <p>Target: {target} Unit</p>
      /* Conditional Rendering: Tampilkan pesan jika target
      tercapai */
      {jumlah >= target ? (
        <div className="alert alert-success d-inline-block">Target
        Tercapai!</div>
      ) : (
        <div className="alert alert-secondary d-inline-
        block">Produksi Berjalan...</div>
      )}
      <div className="mt-3">
        <button className="btn btn-primary me-2"
        onClick={tambahProduksi}>
          +1 Unit (Sensor OK)
        </button>
        <button className="btn btn-danger" onClick={reset}>

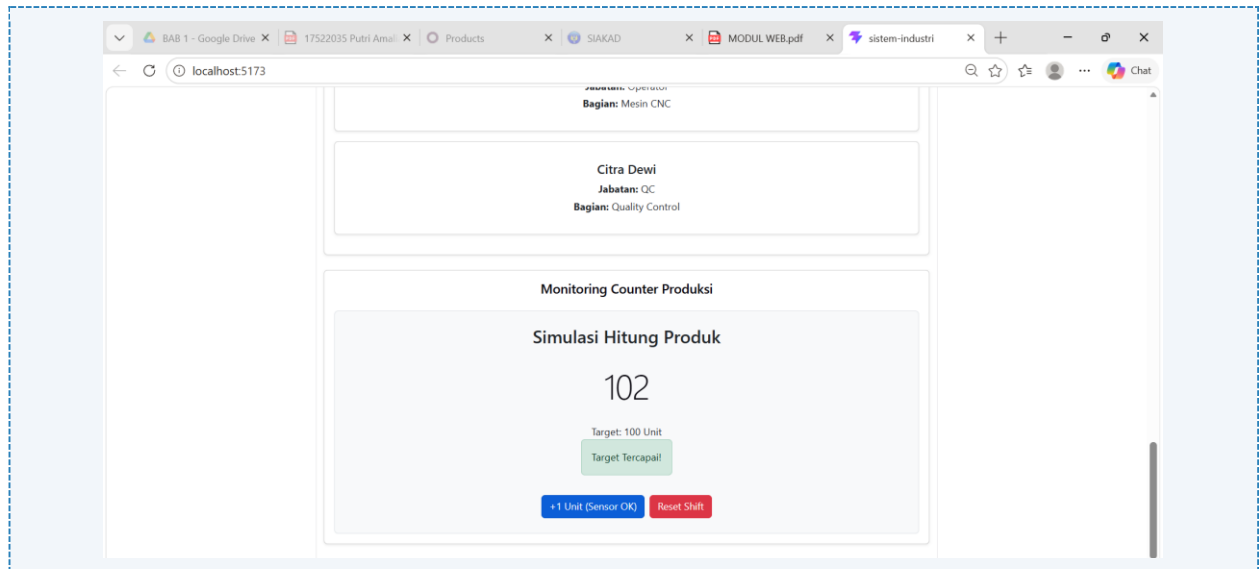
```

```

        Reset Shift
      </button>
    </div>
  </div>
);
}
export default CounterProduksi;

```

Screenshot Hasil:



Gambar E.1 – Komponen Counter Produksi

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

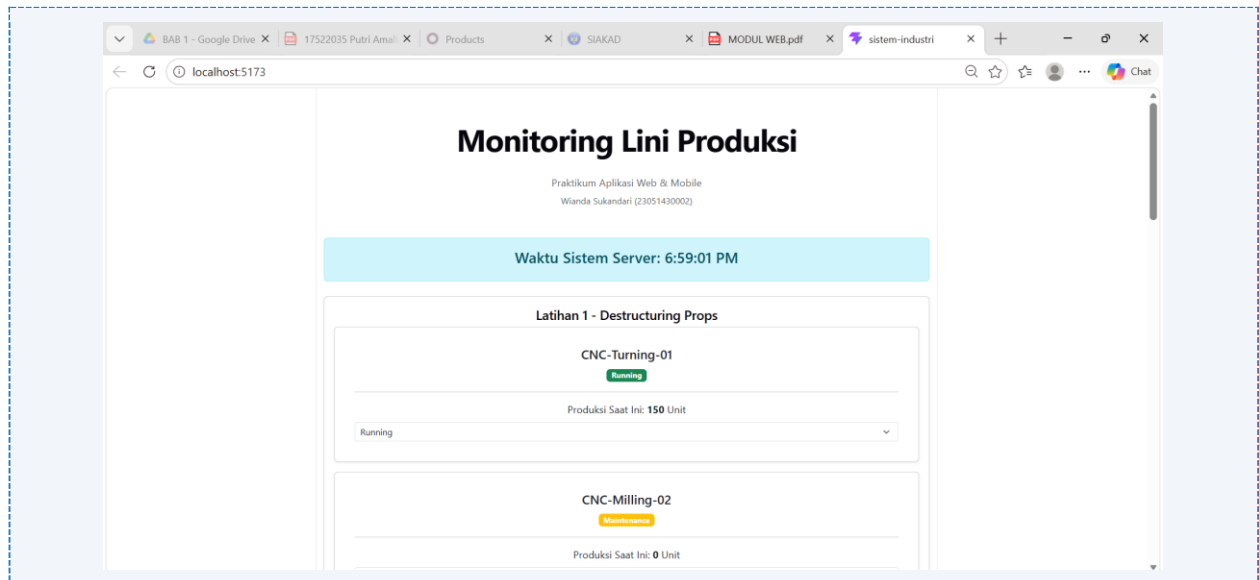
```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React, { useState, useEffect } from 'react';
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  // useEffect berjalan sekali setelah komponen dirender pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap detik
    }, 1000);
    // Cleanup function: Dijalankan saat komponen dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali saat mount
  return (
    <div className="alert alert-info text-center">
      <h4>Waktu Sistem Server: {waktu.toLocaleTimeString()}</h4>
    </div>
  );
}

```

```
);
}
export default JamDigital;
```

Screenshot Hasil:



Gambar E.2 – useEffect untuk Real-time Clock

E.3 Langkah 3 – Contoh Form Input State

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React, { useState } from "react";

// 2. Komponen Fungsi
function KartuMesin(props) {
  // Menerima data dari props
  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;

  // State lokal untuk status
  const [statusLokal, setStatusLokal] = useState(status);

  // Logika penentuan warna badge berdasarkan statusLokal
  let badgeColor = "bg-secondary";
  if (statusLokal === "Running") badgeColor = "bg-success";
  if (statusLokal === "Stop") badgeColor = "bg-danger";
  if (statusLokal === "Maintenance") badgeColor = "bg-warning";

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{namaMesin}</h5>
```

```

<span className={`badge ${badgeColor}`}>{statusLokal}</span>

<hr />

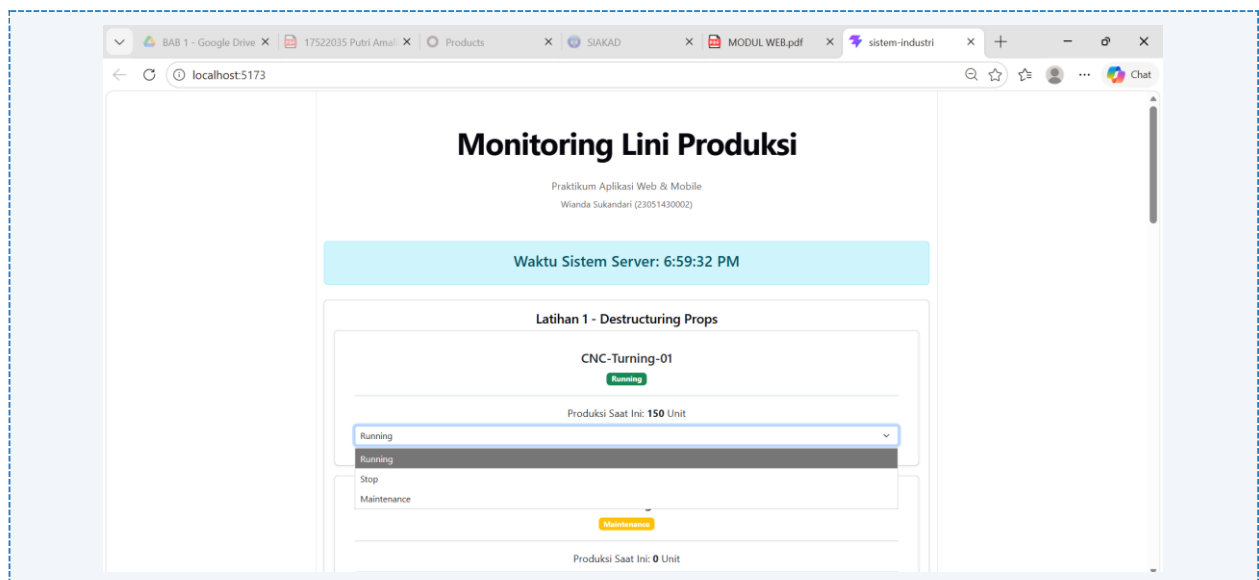
<p>
  Produksi Saat Ini: <strong>{produksi}</strong> Unit
</p>

<div className="mt-2">
  <select
    className="form-select form-select-sm"
    value={statusLokal}
    onChange={ (e) => setStatusLokal(e.target.value)}
  >
    <option value="Running">Running</option>
    <option value="Stop">Stop</option>
    <option value="Maintenance">Maintenance</option>
  </select>
</div>
</div>
</div>
);
}

// 3. Export
export default KartuMesin;

```

Screenshot Hasil:



Gambar E.3 – Form Input State

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from "react";
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());

  // LATIHAN 1:
  const [kota, setKota] = useState(""); // Menambahkan state kota
  // useEffect berjalan sekali setelah komponen dirender pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap detik
    }, 1000);
    // Cleanup function: Dijalankan saat komponen dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali saat mount

  // Menambahkan untuk Latihan 1
  useEffect(() => {
    document.title = `Jam ${kota}`;
  }, [kota]);

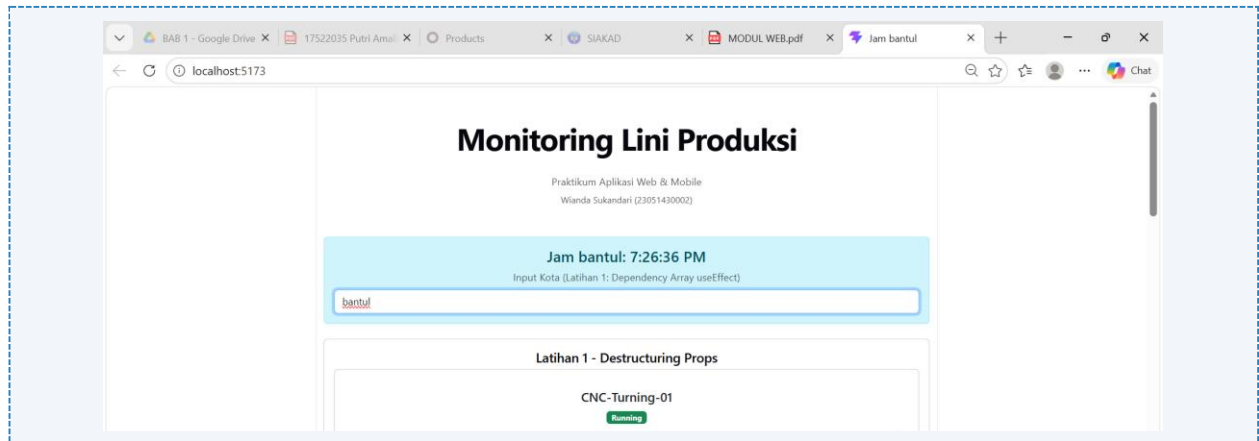
  return (
    <div className="alert alert-info text-center">
      <h4>
        Jam {kota || "Indonesia"}: {waktu.toLocaleTimeString()}
      </h4>

      <p className="mt-2 mb-1 text-muted">
        Input Kota (Latihan 1: Dependency Array useEffect)
      </p>

      <input
        type="text"
        placeholder="Masukkan nama kota"
        className="form-control mt-2"
        value={kota}
        onChange={(e) => setKota(e.target.value)}
      />
    </div>
  );
}
```



```
    </div>
  );
}
export default JamDigital;
```



Gambar F.1 – Dependency Array useEffect

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from 'react';

function CounterProduksi() {
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  const [status, setStatus] = useState("Produksi Berjalan");

  // Tambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };

  // Reset
  const reset = () => {
    setJumlah(0);
    setStatus("Produksi Berjalan");
  };

  // Emergency Stop
  const emergencyStop = () => {
```

```
setStatus("EMERGENCY");
};

return (
  <div className="text-center p-4 border rounded bg-light">

    <h3>Simulasi Hitung Produk</h3>

    <h1 className="display-4">{jumlah}</h1>

    <p>Target: {target} Unit</p>

    {/* Conditional Rendering */}
    {status === "EMERGENCY" ? (
      <div className="alert alert-danger d-inline-block">
        ⚠ EMERGENCY STOP! Produksi dihentikan
      </div>
    ) : jumlah >= target ? (
      <div className="alert alert-success d-inline-block">
        Target Tercapai!
      </div>
    ) : (
      <div className="alert alert-secondary d-inline-block">
        Produksi Berjalan...
      </div>
    )}

    <div className="mt-3">

      <button
        className="btn btn-primary me-2"
        onClick={tambahProduksi}
        disabled={status === "EMERGENCY"}
      >
        +1 Unit
      </button>

      <button
        className="btn btn-danger me-2"
        onClick={emergencyStop}
      >
        Emergency Stop
      </button>

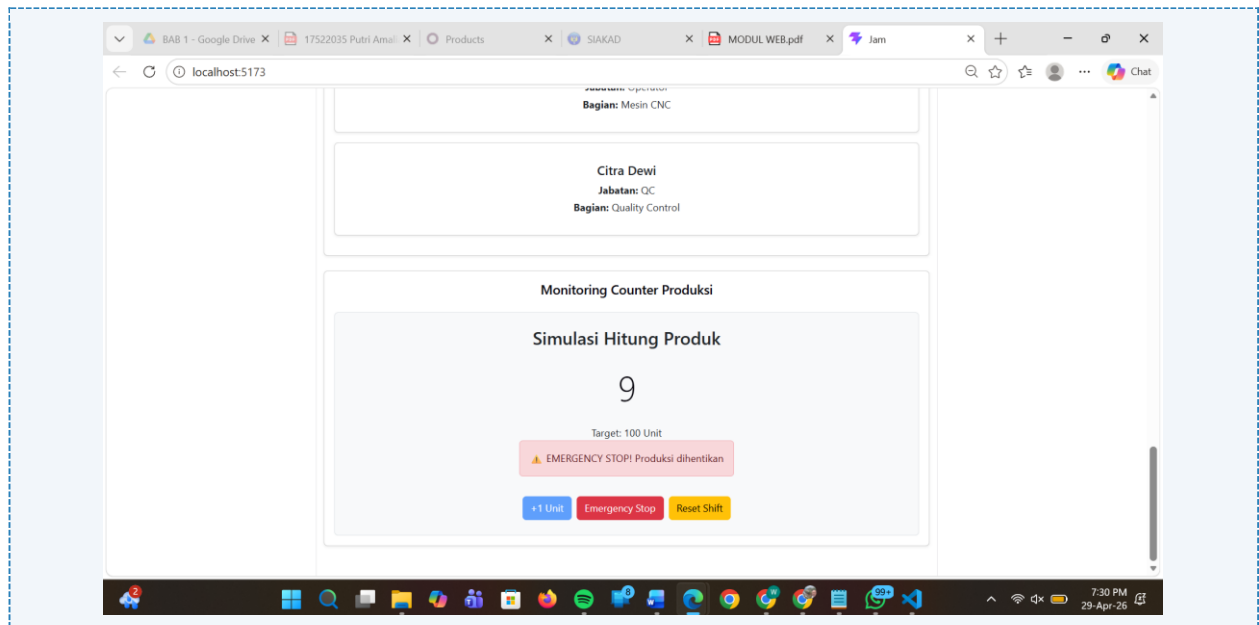
      <button
        className="btn btn-warning"
        onClick={reset}
      >
        Reset Shift
      </button>

    </div>

  </div>
);
```

```
    </div>
  </div>
);
}

export default CounterProduksi;
```



Gambar F.2 – Conditional Rendering

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- Buat komponen form input: 'Plan Time', 'Run Time', 'Total Parts', 'Good Parts'.
- Gunakan State untuk menyimpan nilai-nilai ini.
- Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
- Rumus OEE = (Availability x Performance x Quality).
- Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, dibuat sebuah Kalkulator OEE (Overall Equipment Effectiveness) sederhana menggunakan React dengan memanfaatkan useState untuk menyimpan data input seperti Plan Time, Run Time, Total Parts, dan Good Parts. Setiap nilai yang dimasukkan pengguna langsung tersimpan ke dalam state sehingga dapat diproses secara real-time tanpa perlu menekan tombol hitung. Perhitungan dilakukan secara otomatis menggunakan rumus OEE yaitu $\text{Availability} \times \text{Performance} \times \text{Quality}$, di mana Availability diperoleh dari Run Time dibagi Plan Time, Performance dari Total Parts dibagi Run Time, dan Quality dari Good Parts dibagi Total Parts. Hasil akhir ditampilkan dalam bentuk persentase agar lebih mudah dipahami.

Selain itu, digunakan conditional rendering untuk memberikan indikator visual pada hasil OEE. Jika nilai OEE kurang dari 50%, maka hasil ditampilkan dengan warna merah sebagai tanda performa rendah. Jika nilai OEE lebih dari 85%, hasil ditampilkan dengan warna hijau sebagai indikator performa sangat baik. Pendekatan ini membuat sistem monitoring menjadi lebih informatif, sederhana, dan mudah digunakan.

Kode Utama:

```
import React, { useState } from 'react';

function OEECalculator() {
  const [planTime, setPlanTime] = useState(0);
  const [runTime, setRunTime] = useState(0);
  const [totalParts, setTotalParts] = useState(0);
  const [goodParts, setGoodParts] = useState(0);

  // Perhitungan OEE
  const availability =
    planTime > 0 ? runTime / planTime : 0;

  const performance =
    runTime > 0 ? totalParts / runTime : 0;

  const quality =
    totalParts > 0 ? goodParts / totalParts : 0;

  const oee =
    availability * performance * quality * 100;

  // Warna hasil
  let warnaHasil = "black";

  if (oee < 50) {
    warnaHasil = "red";
  } else if (oee > 85) {
    warnaHasil = "green";
  }

  return (
    <div className="card p-4 shadow-sm">
      <h2 className="text-center mb-4">
        Kalkulator OEE
      </h2>

      <div className="mb-3">
```

```
    <label>Plan Time</label>
    <input
      type="number"
      className="form-control"
      value={planTime}
      onChange={ (e) => setPlanTime (Number (e.target.value)) }
    />
  </div>

  <div className="mb-3">
    <label>Run Time</label>
    <input
      type="number"
      className="form-control"
      value={runTime}
      onChange={ (e) => setRunTime (Number (e.target.value)) }
    />
  </div>

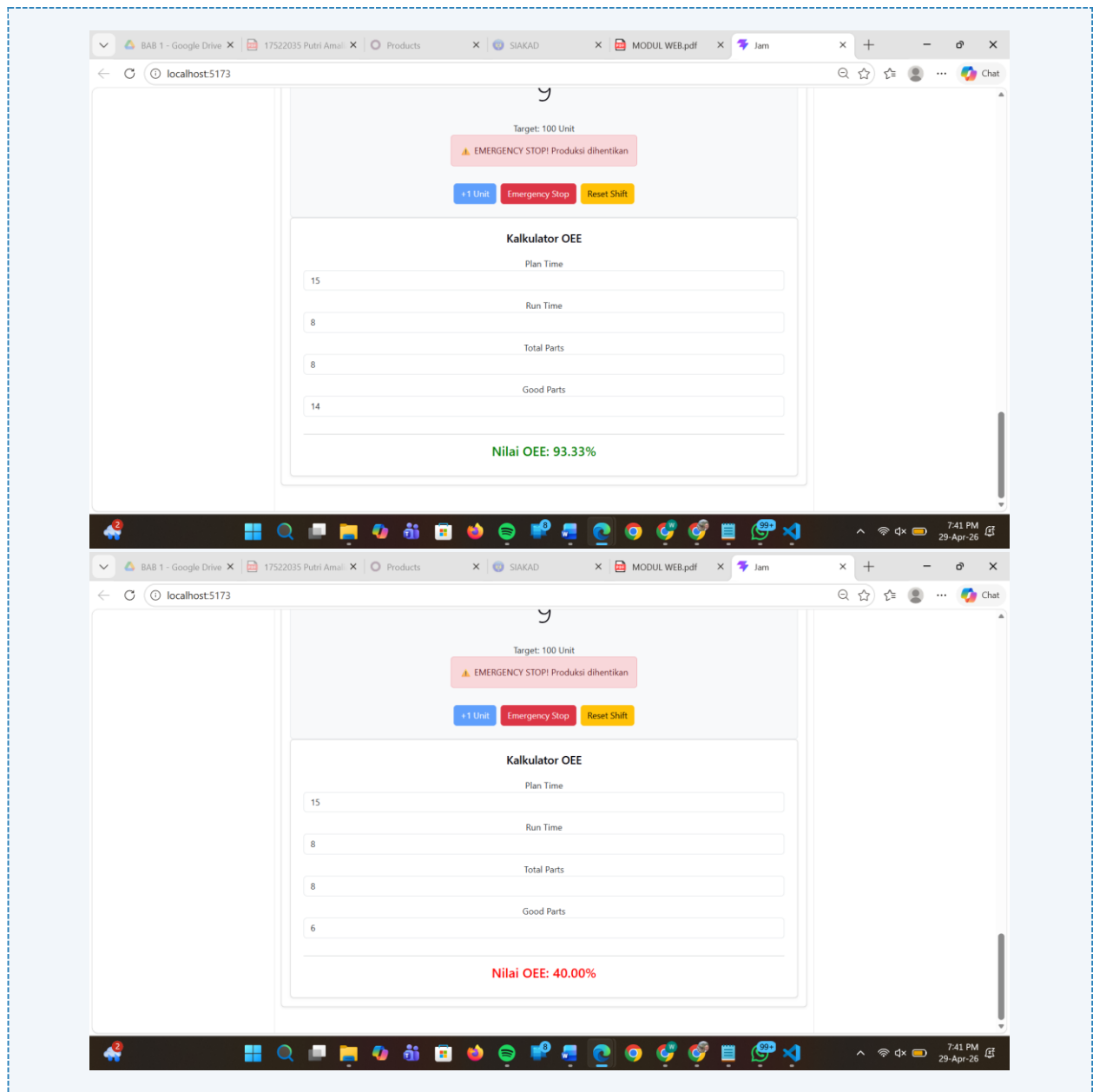
  <div className="mb-3">
    <label>Total Parts</label>
    <input
      type="number"
      className="form-control"
      value={totalParts}
      onChange={ (e) => setTotalParts (Number (e.target.value)) }
    />
  </div>

  <div className="mb-3">
    <label>Good Parts</label>
    <input
      type="number"
      className="form-control"
      value={goodParts}
      onChange={ (e) => setGoodParts (Number (e.target.value)) }
    />
  </div>

  <hr />

  <h4 style={{ color: warnaHasil }}>
    Nilai OEE: {oee.toFixed(2)}%
  </h4>
</div>
);
}

export default OEECalculator;
```



Gambar F.3 – Kalkulator OEE Sederhana

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori React yang dipelajari, khususnya pada penggunaan `useState`, `useEffect`, `dependency array`, `conditional rendering`, `props`, dan `default props`. Pada komponen `JamDigital`, penggunaan `useEffect` dengan `dependency array` [`kota`] berhasil membuat `document.title` berubah secara otomatis mengikuti nama kota yang diinput, misalnya menjadi “Jam Sleman”. Hal ini menunjukkan bahwa efek hanya berjalan saat state kota berubah, sesuai konsep `dependency array` pada React.

Pada komponen `CounterProduksi`, penggunaan `useState` berhasil mengatur jumlah produksi, target produksi, dan status `emergency`. Saat tombol “+1 Unit” ditekan, nilai produksi bertambah satu per satu secara `real-time`. Ketika tombol “Emergency Stop” ditekan, status berubah menjadi “EMERGENCY”, tombol “+1” menjadi tidak aktif (`disabled`), dan pesan peringatan berwarna merah muncul. Hal ini sesuai dengan teori `conditional rendering`, yaitu tampilan berubah berdasarkan kondisi state tertentu.

Pada latihan `props`, komponen `KartuMesin` berhasil menerima data mesin melalui `props` seperti nama, status, dan jumlah produksi. Selain itu, pada latihan `default props`, ketika nilai produksi tidak diberikan, sistem otomatis menampilkan nilai default yaitu 0. Ini menunjukkan bahwa konsep `props` dan `default props` berjalan dengan baik sesuai teori.

Pada mini project `Kalkulator OEE`, sistem mampu menghitung nilai OEE secara otomatis setiap kali input berubah tanpa perlu tombol hitung tambahan. Perhitungan `Availability`, `Performance`, dan `Quality` berjalan sesuai rumus teori OEE, dan hasil akhir ditampilkan dalam bentuk persentase dengan warna indikator berdasarkan nilai OEE. Hal ini membuktikan bahwa penerapan `state management` dan `real-time calculation` berjalan sesuai konsep yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Komponen tidak tampil di halaman karena belum dipanggil di <code>App.jsx</code>	Menambahkan import komponen dan memanggilnya di bagian <code>return App.jsx</code> agar komponen dapat dirender
Error karena penulisan path file salah, seperti folder <code>Komponen</code> atau <code>Latihan</code> tidak sesuai	Memeriksa kembali struktur folder project dan menyesuaikan path import agar sesuai lokasi file sebenarnya
Tampilan hasil latihan terlihat double karena <code>Latihan 1</code> dan <code>Latihan 2</code> memiliki output yang mirip	Menggabungkan beberapa komponen ke dalam satu halaman utama dan merapikan layout agar lebih terstruktur dan tidak terlihat berulang

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini sangat relevan dengan penerapan Teknik Industri, terutama dalam sistem monitoring produksi dan pengambilan keputusan berbasis data secara real-time. Penggunaan React untuk membuat dashboard monitoring dapat membantu perusahaan dalam memantau kondisi mesin, status produksi, serta performa operator secara lebih cepat dan efisien.

Sebagai contoh, pada industri manufaktur seperti pabrik otomotif atau makanan, supervisor produksi perlu mengetahui status mesin seperti Running, Maintenance, atau Stop secara langsung. Dengan komponen seperti KartuMesin, informasi tersebut dapat ditampilkan secara visual sehingga memudahkan pengawasan lini produksi.

Penggunaan CounterProduksi juga dapat diterapkan untuk menghitung output produksi harian secara real-time. Jika terjadi kondisi darurat seperti kerusakan mesin, tombol “Emergency Stop” dapat digunakan untuk menghentikan proses dan memberikan peringatan kepada operator agar tindakan cepat dapat dilakukan.

Selain itu, mini project OEE sangat penting dalam industri karena OEE merupakan salah satu indikator utama untuk mengukur efektivitas mesin. Dengan sistem OEE berbasis web, perusahaan dapat mengetahui apakah performa mesin masih optimal atau perlu dilakukan perbaikan. Misalnya, jika nilai OEE berada di bawah 50%, maka tim maintenance dapat segera melakukan analisis penyebab downtime untuk meningkatkan efisiensi produksi.

Dengan demikian, modul ini tidak hanya melatih kemampuan pemrograman web, tetapi juga mendukung penerapan konsep Teknik Industri dalam pengendalian produksi, evaluasi performa mesin, dan peningkatan produktivitas perusahaan.

H. KESIMPULAN

1	Praktikum ini membantu memahami konsep dasar React seperti useState, useEffect, props, default props, dependency array, dan conditional rendering dalam pembuatan aplikasi web interaktif.
2	Komponen JamDigital berhasil menunjukkan penggunaan useEffect dengan dependency array untuk mengubah document.title secara otomatis berdasarkan input nama kota secara real-time.
3	Komponen CounterProduksi berhasil menerapkan conditional rendering melalui fitur tombol “Emergency Stop”, di mana status sistem berubah, tombol produksi dinonaktifkan, dan pesan peringatan ditampilkan.
4	Mini project Kalkulator OEE berhasil menghitung nilai efektivitas mesin secara otomatis berdasarkan input produksi, sehingga dapat digunakan sebagai simulasi monitoring performa mesin dalam industri.
5	Penerapan konsep React pada modul ini sangat relevan dengan bidang Teknik Industri, khususnya dalam monitoring lini produksi, evaluasi performa mesin, dan pengambilan keputusan berbasis data secara real-time.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Documentation. (2026). <i>Using the State Hook</i> . Retrieved from https://react.dev/reference/react/useState
[3]	React Documentation. (2026). <i>Using the Effect Hook</i> . Retrieved from https://react.dev/reference/react/useEffect
[4]	Mozilla Developer Network. (2026). <i>JavaScript Guide</i> . Retrieved from https://developer.mozilla.org/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
	
Wianda Sukandari NIM: 23051430002	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____